

SQL Questions & Solutions

SQL Question 1 — Days Between Customer Orders

Marks: 20

Database: CUSTOMERS_ORDERS_DB

For each order placed by a customer, calculate the number of days that have passed since the customer's immediately previous order. Use the LAG() window function to retrieve the previous order date for the same customer.

Requirements:

- Process the orders separately for each customer using customer_id.
- Use LAG() to retrieve the immediately previous order_date for each customer.
- Determine the sequence using order_date and then order_id.
- Calculate the number of days between the current order date and the previous order date.
- For the first order of each customer, days_since_previous_order should be NULL.
- Sort by customer_name, order_date, and order_id.

Required Output Columns: customer_name, order_id, order_date, days_since_previous_order

SQL Query:

```
</> SQL

SELECT
    c.customer_name, o.order_id, o.order_date,
    DATEDIFF(
        o.order_date,
        LAG(o.order_date) OVER (
            PARTITION BY o.customer_id
            ORDER BY o.order_date, o.order_id
        )
    ) AS days_since_previous_order
FROM Customers c
JOIN Orders o ON c.customer_id = o.customer_id
ORDER BY c.customer_name, o.order_date, o.order_id;
```

SQL Question 2 — Event Average Finish Time

Marks: 20 **Difficulty:** Medium

Database: ATHLETICS_RESULTS_DB

Generate a report that, for every recorded result, displays the event name, athlete name, athlete's finish time in seconds, and the average finish time of all recorded performances within the same event.

Requirements:

- Use the AVG() window function with PARTITION BY to calculate the average separately for each event.
- Only results where status = 'Recorded' should be included.
- Disqualified and DNS results must not appear and must not be included in the event average.
- Round the calculated average finish time to 2 decimal places.
- Sort by event_name, finish_time, and athlete_name.

Required Output Columns: event_name, athlete_name, finish_time, avg_event_time

SQL Query:

```
</> SQL

SELECT
    e.event_name, a.athlete_name, r.finish_time,
    ROUND(AVG(r.finish_time) OVER (PARTITION BY r.event_id), 2) AS avg_event_time
FROM Results r
JOIN Events e ON r.event_id = e.event_id
JOIN Athletes a ON r.athlete_id = a.athlete_id
WHERE r.status = 'Recorded'
ORDER BY e.event_name, r.finish_time, a.athlete_name;
```